

SAFESCALE

A hybrid multi-agent and multi-objective system for SAFE PI Planning

NSGA-II, Critical Chain and conditional HITL optimization

Gabriel Leulmi

Master's Thesis, 2025-2026

May 2026

Abstract

Planning a *Program Increment* (PI, a delivery cycle of eight to twelve weeks) within the Scaled Agile Framework [21] is a **resource-constrained scheduling problem**: one must decide what to do, when, and by which team, while respecting the available skills, the dependencies and the business deadlines. This type of problem (called RCPSP in operations research [20, 19]) is **non-polynomial**: the number of possible schedules explodes so fast with project size that no computer can test them all. Planning therefore relies today on human experience. The *Release Train Engineer* (RTE, facilitator of the delivery train) runs the ritual, but several other SAFE roles are involved: *Scrum Master*, *Product Owner* (PO), *Chief Product Owner* (CPO), *Engineering Delivery Managers* and the *Development Teams* that provide estimates and *refinement*. Market tools serve to capture and visualize these exchanges, not to explore several candidate scenarios or to evaluate them against multiple criteria. This thesis presents **SAFESCALE**, a decision-support prototype that assembles three elements. First, an optimization by **multi-objective genetic algorithm** (NSGA-II [23, 7], which evolves a population of candidate schedules over several generations), extended to six objectives including a **WSJF-weighted tardiness** (*Weighted Shortest Job First*, the economic priority score of an undertaking): the delay of each feature in days, multiplied by that business value, inspired by a multi-project industrial case [14], with a **local search** that refines the best schedules (“memetic” approach [3]) and starting schedules built by domain heuristics (inspired by INSGA-II, for *Improved NSGA-II* [1, 2]). Second, a multi-agent architecture *orchestrated* in five steps, with a human in the loop in a **conditional** manner (consulted only when a problem is detected) and an explicit trace of decisions. Finally, a **safety-margin heuristic**, inspired by the principle of Goldratt’s *Critical Chain* [24] and by Kingman’s law [25]: a deterministic risk score, computed by a rule rather than learned from data, inserts reserve time after the riskiest *stories*. The system accounts for the effort-duration / calendar-duration ratio (a story point is rarely executed in as many days as it is worth), team specialization (*one team owns one feature* [21]) and capacity bottlenecks per skill. It is instantiated on a reproducible synthetic dataset (one ART, nine teams, four three-week sprints, 112 to 131 *stories* depending on the draw) whose dimensions are inspired by an interview with a Product Owner.

Keywords. SAFE ; PI Planning ; RCPSP ; memetic NSGA-II ; orchestrated multi-agent system ; conditional HITL ; safety margins inspired by the *Critical Chain* ; Kingman’s law ; WSJF-weighted tardiness.

1 Introduction

1.1 Context

The *Scaled Agile Framework* (SAFE) defines the planning of an *Agile Release Train* (ART, see appendix A) gathering 5 to 15 teams and 50 to 125 people, organized around a central ceremony, the **PI Planning**, over one to two days to plan the following eight to twelve weeks [21].

Four terms structure the whole document and are worth stating right away. An **epic** designates a large undertaking, for instance reworking a payment module. It breaks down into **features**, units deliverable within one PI. Each feature in turn breaks down into **stories**, the unit a team estimates and then carries out. The effort of a story is expressed in **story points**, a relative measure rather than a number of days. Finally, the **WSJF** (*Weighted Shortest Job First*) is the economic priority score attached to an undertaking: the higher it is, the more costly it is to delay its delivery.

The ritual mobilizes several roles whose responsibilities

are precisely distributed:

- the *Release Train Engineer* (RTE) runs the ritual and arbitrates dependencies between teams;
- the *Chief Product Owner* (CPO) prioritizes the *features* in his scope and distributes them across teams;
- each *Product Owner* (PO) carries the content of his team and presents its proposals;
- the *Scrum Master* (SM) facilitates the ceremony on the team side;
- the *Engineering Delivery Managers* (EDM) and the other stakeholders accept or refuse the proposed commitments;
- the *Development Teams* produce the estimates and the *refinement* that feed all decisions.

None of these roles today has a tool to explore several candidate scenarios and evaluate them quantitatively; trade-offs are made by eye, on the basis of experience.

1.2 Motivation

Formally, the RTE’s task falls under resource-constrained scheduling: placing activities in time while

people are in limited supply, skills are not interchangeable, some activities must wait for others, and several projects compete for the same resources. The literature designates this class of problems as the *Multi-Skill Resource-Constrained Multi-Project Scheduling Problem* (MS-RCMPSP), an extension of the RCPSP [20]. It is NP-hard from about sixty activities onwards [26, 19]: beyond that size, no known algorithm guarantees the optimum in polynomial time, and one must settle for approximate solutions obtained by exploration. “Optimizing” here means three concrete things: (i) generating several candidate PI scenarios instead of a single one, (ii) evaluating them simultaneously against several criteria, and (iii) recommending a compromise while keeping the final decision with the human. This complexity is not addressed by existing tools, which remain focused on capturing and visualizing SAFe artifacts. Optimization research has nonetheless worked extensively on neighboring problems: hybrid variants of NSGA-II [3, 4] and NSGA-III [9, 11] give good results on instances of RCPSP and job-shop scheduling. The purpose of this work is to transpose these approaches to the SAFe domain, with its own specificities (WSJF, sprints, specialized teams, *readiness* status).

1.3 Contributions

The thesis proposes six **original** contributions of the SAFESCALE system, each transposing or extending a result published in the literature to the specific context of SAFe PI Planning:

- C1.** A **sixth evaluation criterion**, WSJF-weighted tardiness: the delay of each feature, multiplied by its business value. The delay of a high-priority undertaking therefore weighs more than that of a secondary one. A transposition to the SAFe context of an objective validated on a multi-project industrial case [14].
- C2.** **Six starting schedules** built from business rules, replacing an entirely random initial population; one of them combines business value and time pressure, adapted from the INSGA-II strategy [1, 2].
- C3.** A **refinement phase** applied to *front 0*, that is, the set of schedules no other one beats simultaneously on every criterion: those schedules are picked up and altered by small targeted touch-ups, instead of merely being recombined at random. Inspired by Memetic NSGA-II [3].
- C4.** An **architecture in five steps** entrusted to specialized components, where human intervention is required only when a problem is detected, rather than at every step. In line with multi-agent architectures coupled with NSGA-II [6].
- C5.** A **safety margin computed story by story**, proportional to a deterministic five-factor risk score, rather than a uniform cushion applied everywhere. A heuristic inspired by the Goldratt principle [24] and Kingman’s law [25]; the five co-

efficients are set by us, not calibrated on data.

C6.

A **complete chain**, from the dataset to the final choice of scenario, replayable identically on a synthetic dataset calibrated on a real ART.

1.4 Outline of the thesis

Section 2 formalizes the problem and specifies the **scope** of the prototype. Section 3 synthesizes the state of the art in five families. Section 4 describes the model and the key parameters. Section 5 presents the architecture of the flow. Section 6 details NSGA-II. Section 7 synthesizes the design decisions and their sources. Section 8 lists the flow logic problems and the corrections made. Section 9 describes the experimental protocol and the campaign’s quantified results. Section 10 presents the qualitative feedback on the solution. Section 11 sets out the limits and what is not taken into account. Section 12 lists the points to explore. Section 13 concludes. Two appendices (A and B) provide a glossary of acronyms and technical terms.

2 Problem statement

2.1 The existing PI Planning process

Before modeling the problem, one must understand how a PI Planning unfolds *today*, without an optimization tool. The following description comes from an interview with a *Product Owner* (PO) operating on a real ART.

Before the PI Planning. Preparation spans several weeks and mobilizes several roles:

- The **CPO** (*Chief Product Owner*) **prioritizes the features** in his scope and makes sure they can reach the *ready* state: for this he organizes **co-engineering workshops** with the teams.
- The **duration of epics** is estimated upstream in order to spread the work over the various upcoming PIs.
- The CPO **distributes the features by team**; each team then indicates which ones it is able to address. It sometimes happens that an epic is **split into features distributed across several teams**.
- Once the features are allocated, in **pre-PI Planning**, each PO prepares the scope of his team and lists the topics to handle.

During the PI Planning. The ceremony itself follows a well-established sequence:

- The **Scrum Master** handles facilitation, the **PO** brings the substance of the discussions.
- The **features** retained for the PI are **listed**.
- Each team performs a **poker planning** after the breakdown of features into *stories*, aiming for a size of **8 SP** (story points) **maximum per story**.
- The teams **propose the features** they declare themselves able to address. It is at this stage that the *readiness* status comes into play: if a team identifies **uncertainties** on some items to develop (unclear specifications, unconfirmed external dependency, etc.), it does not **commit** to them for the upcoming PI (in SAFe vocabulary, the team does not

commit: it declines to undertake delivery, for lack of sufficient visibility), and may propose to **push them to the next PI**.

During execution. Planning remains **deliberately general**: at each sprint, the teams hold a **readjustment meeting** to correct the plan according to on-the-ground reality.

Limits of this process. This sequence is mature and proven, but it remains entirely **manual**: the features→teams distribution, the detection of overloads and dependency conflicts, the arbitration between competing priorities all rely on human experience. No **algorithmic exploration** of alternative scenarios is possible, and no **quantitative multi-criteria evaluation** assists the decision. It is precisely this gap that SAFESCALE seeks to fill, by inserting itself as a decision aid **without modifying the existing roles** (CPO, PO, Scrum Master, RTE).

2.2 What makes this problem hard

PI Planning combines five characteristics which, taken together, put the problem out of reach of an exact resolution [20, 19]:

- several undertakings share the same people, and what is granted to one is taken away from another (*multi-project*) [16];
- people are not interchangeable: each masters only a subset of skills, and some skills are held by no more than a handful of individuals (*multi-skill*) [15];
- six criteria must be satisfied at once although they contradict each other: total duration, delay risk, safety margin, overload of individuals, a team exceeding its capacity on a sprint, and delay weighted by business value (*multi-objective*, detailed in section 6) [7, 19];
- a feature is owned by a single team, a principle SAFe states as “*one team owns one feature*” [21];
- a dependency declared between two features constrains all of their stories, not only the two directly concerned [14].

2.3 Five families of recurring conflicts

The interview conducted with a Product Owner, complemented by our modeling, reveals five recurring families of conflicts. Their manual resolution, which mobilizes all the SAFe roles (RTE, CPO, PO, Scrum Master, EDM and dependent teams), is what we seek to instrument:

1. **Capacity exceeded**: a team is assigned more work than it can absorb (CAPACITY_VIOLATION).
2. **Dependency violated**: a story is scheduled before the one it depends on (DEPENDENCY_BREACH).
3. **Scope overflow**: features the ART cannot absorb in the current PI and that must be deferred (FEATURE_OVERFLOW).
4. **Deadline at risk**: a feature ends after the date set for it (DEADLINE_RISK).
5. **Unbalanced load**: more than 30 % load difference between two teams (LOAD_IMBALANCE).

These five conflicts are *interdependent*: resolving an

overload often creates a dependency violation, which creates a *deadline risk* [14].

2.4 Research question

Can one design a hybrid system combining multi-objective optimization, multi-agent architecture and a risk model, to automate the generation and evaluation of SAFe PI Planning scenarios, while keeping for the human an effective decision-making role?

2.5 Scope of the prototype

The prototype focuses on a precisely defined case study, in order to guarantee the *reproducibility* and legibility of the results:

- **A single ART**, nine specialized teams (*backend, frontend, data, devops, mobile, qa, fullstack fallback*), in line with the 5-15 teams per ART range [21].
- **A PI of four three-week sprints** (60 working days), a format noted during the PO interview and consistent with the 8-12 weeks range [21].
- **About fifty resources** (5-7 per team, 52 on the reference dataset), 9 epics, ≈ 2 features per epic (≈ 18 features), 5 to 10 *stories* per feature, that is 112 to 131 *stories* depending on the draw (section 9). The *story* is the planning unit: there is no *task* level, in line with the real process observed (section 2.1).
- **Effort→duration conversion**: the empirical value varies from one team to another, typically between **1 and 2 working days per story point**. We use **1 SP = 1.7 days** as a pivot value, allowing a per-team draw in the interval [1.0; 2.0].
- **Synthetic data** generated with a fixed *seed* (*seed=42*), guaranteeing full reproducibility of the experiments without access to confidential real data.
- **Batch mode**: a whole PI is planned in a single end-to-end session, triggered by the RTE. “Batch” does not mean “without interaction”: the five steps of the flow include conditional HITL points where the RTE intervenes during execution (cf. section 5). It is the *continuous-flow replanning* between two PIs (for example in reaction to an external event) that is out of scope.

This scope fixes the framework of the work. The elements that fall outside it (real Jira data, simultaneous multi-ART, real-time *replanning*, reinforcement learning) are explicitly deferred to the limits (section 11) and perspectives (section 12).

3 State of the art

The state of the art is organized in five families covering the axes mobilized by the system: multi-objective evolutionary algorithms, extension to more than three objectives, hybrid and multi-project RCPSP, official SAFe practices, and the integration of machine learning into MOEAs.

3.1 Family 1, NSGA-II and its variants

NSGA-II [23] remains, twenty years after its publication, “by far the most popular *metaheuristic* for multi-objective optimization problems” [7]. Three recent axes of improvement structure our contribution:

Axis 1, Targeted initialization. *INSGA-II* [1] demonstrates empirically on 24 instances that a random initialization strongly degrades convergence; the objective-backed strategies (WSJF, deadline-first, critical path, load-balance) significantly improve the optimization rate [1, 2]. Our seed C2 “WSJF / deadline urgency” extends this idea by combining business value and time pressure.

Axis 2, Memetic hybridization. NSGA-II progresses by random “big jumps” (crossovers and mutations): it explores very broadly, but never *refines* a solution. It may pass right next to an optimum without stopping there. The memetic idea consists of adding to NSGA-II a second complementary mechanism, *greedy local search*: one takes the best solutions of the moment and applies to them **small targeted touch-ups** (move a story from one team to another, swap the order of two stories), keeping only those that improve an objective. The table below illustrates the difference:

Without search	local	With (memetic)
One explores broadly at random, but polishes no solution.	explores at random, but polishes	One explores broadly at random, then polishes the best ones with small modifications.
Good coverage, average fine quality.		Good coverage <i>and</i> fine quality.

Memetic NSGA-II [3] applies this recipe to the best individuals of front 0; on job-shop scheduling problems (*flexible job shop*), the authors obtain fronts of higher quality than those of an exact solver, for a much lower computation time. This is the basis of our contribution C3.

Axis 3, NSGA-II + Q-Learning. In classical NSGA-II, the **moment** at which each task starts is chosen by *fixed* rules (“as early as possible once dependencies are satisfied”, for example). The idea of this axis is to replace this frozen rule with a **learned policy**: a *reinforcement learning* agent tries different start dates over many schedules, receives feedback “it was a good or a bad idea” depending on the quality of the final schedule, and progressively *learns* which dates work best. [4] formalizes this as a *Markov decision process*, solved by the *Q-Learning* algorithm. The table below illustrates the difference:

Without Learning	Q-Learning	With (NSGA-II + Q-Learning)
Start dates chosen by a fixed rule, identical on all schedules.		Start dates chosen by a policy <i>learned</i> on hundreds of previous schedules.
No training needed, but quality limited by the rule.		Costly to train, but better final quality.

The published results surpass NSGA-II alone, as well as two other families of nature-inspired algorithms: par-

ticle swarm optimization (PSO) and ant colony optimization (ACO). Training is costly but the approach is rich in information; we identify it as perspective P3.

What we retain and what is missing: these three axes validate separately targeted initialization, local search and timing learning, but none of these works handles a problem structured like SAFe (epic/feature/story hierarchy, *one team owns one feature* constraint, WSJF). They remain on generic *job shop* test benches; the transposition to PI Planning is not immediate and constitutes precisely a part of our work.

3.2 Family 2, NSGA-III for 5+ objectives

With six objectives, the *crowding distance* of NSGA-II becomes less effective at preserving the uniformity of the front [9]. NSGA-III replaces this mechanism with a system of **reference points** [11, 10], and obtains better fronts than NSGA-II on the RCPSP benchmarks of [11].

What we retain and what is missing: NSGA-III is clearly more suited to our number of objectives, and this is why it appears in perspective P2. But it introduces an additional parameterization, the placement of reference points, which requires a study of its own. Starting with NSGA-II, better documented and simpler to debug, is a deliberate prototype choice, not an end in itself.

3.3 Family 3, hybrid and multi-project RCPSP

The reference *Hybrid Genetic Algorithm* [12] remains a classical basis: it is a standard *genetic algorithm* onto which a *repair-based decoding* is grafted (cf. section 6) to make the *mutation* always valid. [13] introduces the concept of *dense genes*, the genes that “saturate” the use of a resource (close to the critical path). The idea is to use these dense genes as *anchor points* during *crossover*, rather than mixing the chromosomes at random: the good parts of a solution are preserved. On industrial multi-project cases, [14] inspired our 6th objective: the WSJF-weighted tardiness (transposition of the *weighted tardiness* weighted by business criticality). For sustainability, [16] adapts NSGA-II to multi-project, and [15] addresses flexible resource profiles, where the number of people assigned to an activity can be adjusted while it runs, by means of a multi-objective swarm algorithm. For duration uncertainty, [17] converts the uncertain (optimistic/pessimistic/likely) into objectives; [18] models costs that vary over time. The review [19] concludes that hybridization generally dominates pure algorithms.

What we retain and what is missing: this family provides the technical building blocks (*decoding*, crossover, tardiness objective) but reasons in abstract “activities”. It ignores two realities of PI Planning: the breakdown into sprints with fixed capacity, and the role of a human decision-maker who decides at the end of the process. Our work reuses the building blocks but recomposes them around these two constraints.

3.4 Family 4, official SAFe and lean flow

The official SAFe source [21] provides the normative parameters: 5-15 teams per ART, 50-125 people, 8-12 weeks per PI, ROAM board for risks. Reinertsen

2009 [25] formalizes the relationship between utilization and *lead-time* (Kingman's law): beyond 80 % utilization, the lead-time becomes super-linear. This is the basis of our blocking 80 % buffer (DEC-016) and of the pivot value 2.0 of the `span_factor`.

What we retain and what is missing: SAFe describes *what* to do (the ceremonies, the roles, the artifacts) but not *how* to optimize the content of a PI. The official documentation remains deliberately methodological and offers no quantitative tool. This is precisely the space that this thesis seeks to occupy.

3.5 Family 5, ML + evolutionary algorithms

This family combines **machine learning** (ML) and **multi-objective evolutionary algorithms** (MOEA), the category to which NSGA-II belongs. The common idea: to use a ML model, learned on the history of projects, to steer the optimization toward solutions with a lower predicted *risk*. The *survey* [22] provides an overview. *ENSGA* (*Enhanced NSGA-II, idem*) is an example: it slides a ML model that predicts the fatigue or the risk of operators into the objective function, and optimizes both the total duration (*makespan*) and these human criteria. The multi-agent NSGA-II [6] for its part handles many-objective problems by *distributing* the computation of the *crowding distance* (cf. section 6) among several agents to better cover the front.

What we retain and what is missing: these works show that a risk model can enter the objective function of a MOEA. However, they often mobilize ML learned on large histories. Lacking such a history, we deliberately chose a *deterministic* risk score (equation 7), less ambitious but reproducible and explainable, a deliberate compromise.

3.6 Synthesis of the positioning

Taken separately, these five families each cover an aspect of the problem; none brings them together for SAFe PI Planning. It is this combination that constitutes the contribution of the thesis, summarized in table 1: (i) SAFe-adapted multi-objective NSGA-II with memetics and *urgency* seed, (ii) orchestrated MAS with conditional HITL and trace, (iii) local safety margins, inspired by the Critical Chain, at the *story* level.

Dimension	Existing	SAFESCALE
Optim.	mono-obj. RCPSP	6-obj. NSGA-II + LS
MAS	BDI / centralized	Orchestrated + HITL
Risk	Absent	Targeted margins + score
Domain	Generic sched.	SAFe (WSJF, sprints)
HITL	Final only	Conditional per step

Table 1: Comparative positioning of SAFESCALE.

4 Model and data

4.1 Modeled SAFe hierarchy

SAFe distinguishes two axes that must not be confused: an axis of **organization** (who works with whom) and an axis of **work decomposition** (what is broken down into what). Our model reflects both separately:

organization: $\underbrace{\text{ART} \rightarrow \text{Team} \rightarrow \text{Resource}}$

decomposition: $\underbrace{(\text{Initiative}) \rightarrow \text{Epic} \rightarrow \text{Feature} \rightarrow \text{Story}}$

The *Initiative* level (in parentheses) exists in SAFe above the epic as a large strategic theme; we mention it for completeness but do not model it, for lack of an effect on the planning of a single PI. Our model stops at the *story* on the decomposition axis, the unit actually estimated and planned in the observed process (section 2.1).

To this are added the skills (*Skill*), the dependencies of *features* and *stories*, and the planning artifacts (*PIPlanning*, *StorySchedule*, *Scenario*, *DecisionTrace*). In line with the real process, there is **no task level**: the *story* is the finest grain, estimated in *story points* (SP) during the poker planning, with a ceiling of 8 SP per story.

4.2 Effort, duration and spread conversion

The effort of a *story* is expressed in *story points*. In practice, a story point represents between 1 and 2 effort-days depending on the team; we retain a pivot value of 1.7 days/SP, adjustable per team to reflect this variability:

$$d_s = SP_s \times \alpha_{\text{team}(s)}, \quad \alpha_{\text{team}} \in [1.0; 2.0], \quad \alpha_{\text{default}} = 1.7 \quad (1)$$

But a story estimated at d_s effort-days is never executed over d_s consecutive calendar days: recurring meetings, *context-switching*, technical blockers spread the work out. We model this by a **spread factor**:

$$\text{calendar}(s) = \lfloor d_s \cdot \text{span_factor}_{\text{team}(s)} \rfloor \quad (2)$$

The `span_factor` has a pivot value of 2.0, but it is not identical for all teams: the generator draws a value in the interval [1.8; 2.2] for each team. That pivot value reflects a result from queueing theory known as Kingman's law [25]: a team spending about half its time on a subject takes roughly twice as many calendar days as the effective working time. A story requiring five days of work therefore occupies about ten calendar days, the remainder being absorbed by meetings, waiting and context switching. The ± 0.2 spread reflects the fact that an experienced team suffers less spread (lower bound) than a junior team (upper bound).

4.3 Team specialization

Each team has a specialization among seven categories (*backend*, *frontend*, *data*, *devops*, *mobile*, *qa*, *fullstack*), in line with the SAFe notion of *Agile Team* [21]. The distribution of skills follows an 80/20 rule: 80 % of a resource's skills come from the core specialty of its team, 20 % may spill over. Rare skills (*data*, *mobile*, *devops*) are *exclusive* to the corresponding teams, which should allow an observable capacity bottleneck to be provoked (cf. section 11).

4.4 Reference synthetic dataset

The generator produces, for `seed=42`, a reproducible dataset with dimensions noted on a real ART (section 2.1):

Entity	Quantity
ART	1
Specialized teams (5-15 SAFe)	9
Resources (5-7 / team)	52
Epics	9
Features (≈ 2 / epic)	≈ 18
Stories (5-10 / feature, ≈ 8 SP)	112-131
PI: 4 sprints $\times$ 3 weeks	60 d

5 Architecture and flow

5.1 Technical view in three layers

The application is structured into a **React + TypeScript frontend** (Tailwind, Recharts), a **Python + Django + DRF** (*Django REST Framework*) **backend**, and a **PostgreSQL database** (figure 1). A central orchestrator (`discussion_flow.py`) drives specialized agents via a synchronous request/response protocol, in the manner of the multi-agent NSGA-II proposed by [6]. The LLMs intervene exclusively as **translators** (*rationale* in natural language), never as decision-makers; they are *mockable* for the reproducibility of the tests.

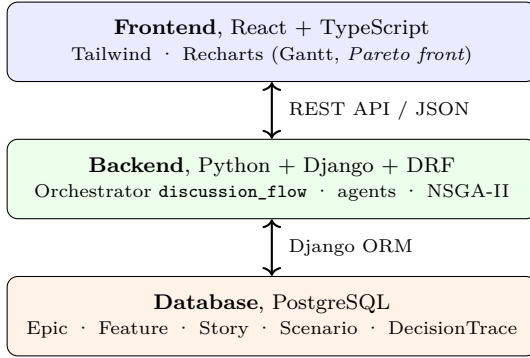


Figure 1: Three-layer architecture of SAFESCALE.

5.2 Five-step flow

The flow is summarized in figure 2:

1. **Preparation** (*readiness*): classification of each feature into `READY` / `BORDERLINE` / `NOT_READY`, in line with the principle of “backlog refinement” [21]. For each team, the *Product Owner* remains the owner of the backlog; he relies on many other actors (CPO, EDM, Scrum Master, dependent teams) who influence the priorities and the state of preparation. HITL if features remain not ready.
2. **Dependencies**: the directed graph of dependencies between features and between stories (the *DAG*) is built, then it is traversed **depth-first** (*Depth-First Search*, DFS) for two purposes: (i) detect dependency cycles, which would make planning impossible; (ii) compute the *critical path*, the longest chain of dependencies that fixes the minimum duration of the PI [14].
3. **Capacity & allocation**: each team builds its capacity plan. **Blocking HITL if a team exceeds 80% of its capacity** [25].
4. **Scheduling**: NSGA-II (*story* $\rightarrow$ resource encoding) produces a Pareto front, from which $K = 5$ **diversi-**

fied scenarios are extracted. Diversification is evaluated by the *Hamming distance* on the feature $\rightarrow$ team assignment: two scenarios are all the more different as they assign more features to distinct teams. The *farthest-first* selection (*cf.* contribution 6) guarantees that each new scenario presented is the farthest possible from the previous ones, which avoids proposing five variants of the same compromise.

5. **Scenario choice: explicit RTE decision**, inspired by the *max-regret* of [4] and the ROAM board of SAFe [21].

5.3 Conditional HITL

Rather than asking at each step “do you want to continue?”, each agent decides locally whether it can decide alone or whether it must open a HITL point. This “conditional” logic is what differentiates SAFESCALE from most existing SAFe tools where the human is either absent, or consulted everywhere.

5.4 Traceability (DecisionTrace)

Each decision is persisted in a `DecisionTrace` model (question asked, decision made, justification, discarded alternatives, *snapshot* of inputs, diff of outputs, human directives, confidence score, LLM *prompt* and response). Multi-level traceability is necessary for the thesis defense supported in the review *Comprehensive Review* on combinatorial NSGA-II [7].

6 NSGA-II: formulation and implementation

6.1 Vocabulary of evolutionary algorithms

NSGA-II belongs to the family of evolutionary algorithms, which proceed by analogy with natural selection. A candidate solution, here a complete schedule, is called an *individual*; the set of solutions examined at the same time forms the *population*; the internal description of an individual is its *chromosome*, cut into *genes*. A generation consists in combining two existing schedules (*crossover*), slightly altering a few of them (*mutation*), then keeping only the best. Repeated over a hundred generations, this cycle moves the population forward without any solution having been built directly.

6.2 Chromosome encoding

An individual is a list $\mathbf{x} = (x_1, \dots, x_n)$ where $x_i = (story_id_i, resource_id_i)$: each gene assigns a *story* to a resource. The order of the genes encodes the execution order. Topological validity is guaranteed at the crossover and mutation by a *repair-based decoding*, a mechanism that, after each genetic operation, traverses the chromosome and automatically reorders the conflicting genes (violated dependency, already busy resource) instead of rejecting the solution, inherited from [12].

6.3 Decoding

Decoding turns a chromosome into a dated schedule. Each story starts as early as possible, that is, as soon as its predecessors are finished and the resource assigned to it becomes free again. Its calendar duration is added, then its safety margin, after which the resource is avail-

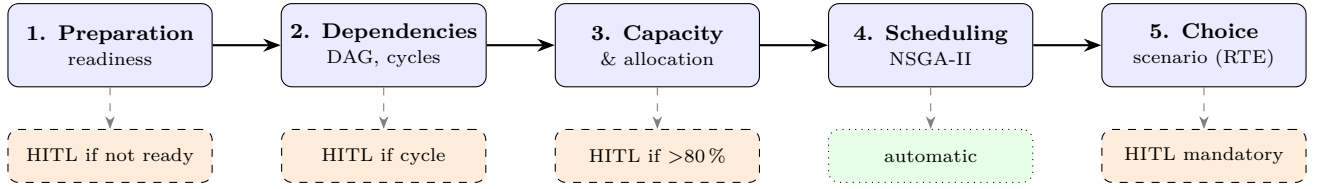


Figure 2: The five-step flow. Step 4 (NSGA-II) is entirely automatic; steps 1, 2 and 3 open a *conditional* HITL point, only if a problem is detected, while step 5 always requires an explicit human decision.

able once more:

$$earliest_i = \max \{ end_p + buffer_p \mid p \prec i \} \quad (3)$$

$$start_i = \max (earliest_i, avail_{r(i)}) \quad (4)$$

$$end_i = start_i + calendar(s_i) \quad (5)$$

$$avail_{r(i)} \leftarrow end_i + buffer_i \quad (6)$$

where $calendar(s_i)$ is the calendar duration of the *story* s_i (equations 1–2) and $buffer_i$ the safety margin defined below. The margin appears twice, and deliberately so: the successor waits for its predecessor to finish **plus** that predecessor’s margin, and the resource only frees up after the same margin. Without this, the margin would exist on paper without ever protecting anyone.

6.4 Safety margin inspired by the Critical Chain

Goldratt’s *Critical Chain* [24] rests on a simple principle: rather than padding every estimate “just in case”, margins that always dissipate, the individual margins are removed and grouped where variance concentrates; the principle is taken up and formalised in fuzzy logic by [17]. We take up that principle, and that principle only.

Two reservations must be stated up front. Goldratt’s method also provides for a **global buffer** at the end of the project, which SAFESCALE does not implement (section 11): our margins are strictly local, story by story. And the way we estimate the variance of a story, below, is a rule we wrote, with coefficients we set. It is therefore a **heuristic inspired by the Critical Chain**, not an implementation of the method.

That rule assigns each *story* a deterministic risk score:

$$risk(s) = 0.30 \cdot \tilde{c} + 0.20 \cdot \tilde{\sigma} + 0.20 \cdot \tilde{p} + 0.20 \cdot \mathcal{K}_{rare}(s) + 0.10 \cdot \tilde{d} \quad (7)$$

$$buffer(s) = \text{round}(risk(s) \cdot 5)$$

where $\tilde{c}, \tilde{\sigma}, \tilde{p}, \tilde{d}$ are respectively the complexity, the **span_factor**, the number of predecessors and the effort (in SP), normalized; $\mathcal{K}_{rare}$ equals 1 if the required skill is *data*, *mobile* or *devops* (underrepresented categories). This modeling extends the idea of [22], which validates empirically the integration of a risk model into the objective function of a MOEA, and that of [17], which converts uncertainty into objectives.

6.5 Six objectives (to minimize)

These six objectives are **not independent**: on the contrary, they contradict one another. If one tries to

compress the *makespan*, one parallelizes more, so the resources are overloaded and the risk rises. If, conversely, one keeps margin to lower the risk, one lengthens the makespan and some features may finish after their deadline. There is therefore no single schedule that is the best on the six criteria at the same time. This is exactly why one uses a multi-objective *evolutionary algorithm*: instead of aiming at a single optimum, NSGA-II builds a *Pareto front*, that is, a set of compromises where each gain on a criterion is paid for elsewhere.

Weighting. During the search, NSGA-II **does not weight** the objectives: all are put on an equal footing via the dominance relation (equation 9). The weighting arrives only at the very end, when a “recommended” scenario must be chosen among those of the front. At that moment, a *composite score* is computed (equation 13) whose weights w_j can be adjusted by the RTE. By default, the six weights are those of the vector (0.22; 0.18; 0.15; 0.13; 0.15; 0.17), in the order of the six objectives listed below.

The six objectives are:

1. Makespan $C_{\max}$ [23].
2. Average risk score $\frac{1}{n} \sum_i risk(s_i)$, computed by the deterministic rule of equation 7 and not by a learned model [22].
3. Buffer deficit (missing margin).
4. Resource overload (parallelism).
5. Capacity violation per team and sprint [21].
6. **WSJF-weighted tardiness** (contribution C1):

$$f_6 = \frac{1}{n} \sum_i WSJF_i \cdot \max(0, end_i - deadline_i) \quad (8)$$

Transposition to the SAFE context of the *weighted tardiness* objective validated on a multi-project industrial case [14].

6.6 Dominance and Pareto front

Strict Pareto dominance [23]:

$$\mathbf{x} \prec \mathbf{y} \iff \forall j, f_j(\mathbf{x}) \leq f_j(\mathbf{y}) \wedge \exists k, f_k(\mathbf{x}) < f_k(\mathbf{y}) \quad (9)$$

When several solutions are equivalent under that relation, some must nonetheless be discarded to keep the population size constant. NSGA-II then keeps the most **isolated** ones first, those with no close neighbour in the objective space, in order to preserve the diversity of the front. The *crowding distance* measures that isolation by summing, on each objective, the gap between the solution’s two immediate neighbours [23]:

$$d_c(i) = \sum_{j=1}^M \frac{f_j(i+1) - f_j(i-1)}{f_j^{\max} - f_j^{\min}} \quad (10)$$

6.7 Six smart seeds

NSGA-II starts with an **initial population** of candidate schedules, typically 40 to 60 individuals. If these schedules are generated completely at random, the algorithm spends the first generations “catching up”. INSGA-II [1, 2] showed that it is much more efficient to **inject a few schedules built by simple heuristics** into the initial population: the evolution then starts near “good regions” of the search space.

Our six generation heuristics each produce a complete initial schedule by ordering the stories according to a business criterion: (1) WSJF descending, business value first; (2) deadline ascending, urgency first; (3) load-balanced, distribute the load fairly; (4) critical path first [14]; (5) minimal risk score first [22]; (6) **WSJF / deadline urgency ratio** (C2) which combines value and time pressure:

$$urgency(f) = \frac{WSJF_f}{\max(1, deadline_f - now)} \quad (11)$$

The rest of the initial population is completed by random schedules to keep diversity.

6.8 Memetic local search (C3)

Every 5 generations, *greedy* local search on the top 15 % of front 0 (resource reassignment, order swap), following [3]. The mutation is kept only if it degrades no objective (non-domination test).

6.9 Selection of the K=5 diversified scenarios

The Pareto front often contains dozens of non-dominated solutions. One cannot present them all to the RTE, and choosing the five “best” does not work: one quickly ends up with five schedules that all resemble one another, because they are neighbors of the same optimum. The review [7] and the multi-agent NSGA-II [6] insist on the importance of a **good coverage** of the front, not only on the raw quality of the retained solutions.

Measuring the difference between two schedules. We use the *Hamming distance*, which counts the number of positions where two schedules differ. Here, each “position” corresponds to a feature, and the stored value is “which team took it”. Two schedules where all the features are allocated to the same teams have a distance of 0; two schedules that assign each feature to a different team are at maximum distance.

Choosing the 5 most different schedules. We start from the scenario recommended by the composite score, then add one by one the four following ones by applying a rule called *farthest-first*: at each step, we take from the front the one that is the **farthest** from the scenarios already retained. This rule is written:

$$\mathbf{x}_{k+1} = \arg \max_{\mathbf{x} \in \mathcal{F}^0 \setminus \mathcal{S}_k} \min_{\mathbf{y} \in \mathcal{S}_k} d_H(\mathbf{x}, \mathbf{y}) \quad (12)$$

$\mathcal{F}^0$ is the entire Pareto front, $\mathcal{S}_k$ the k scenarios already chosen, and d_H the Hamming distance. At each addition, we guarantee that the new scenario is different from all the previous ones, which avoids the five variants of the same compromise.

Composite score (recommended). To designate the “recommended” scenario by default, each objective is normalized between 0 and 1 and a weighted average is made of them:

$$score(\mathbf{x}) = \sum_{j=1}^6 w_j \cdot \frac{f_j(\mathbf{x}) - f_j^{\min}}{f_j^{\max} - f_j^{\min}} \quad (13)$$

$f_j^{\min}$ and $f_j^{\max}$ are the bounds observed on the front; the w_j are the weights of the six objectives. They default to (0.22; 0.18; 0.15; 0.13; 0.15; 0.17) and sum to 1: duration weighs most, overload of individuals least. These values were set by us and have not been validated experimentally; they are adjustable by the RTE and belong to the assumed choices (section 9.9).

6.10 Removal advice at step 3

When a team exceeds 80 % of load [25], a deterministic impact score is computed:

$$impact(f) = WSJF_f \cdot (1 + downstream_blocks_f) \quad (14)$$

The score measures what is lost by removing the *feature*: its business value, multiplied by the number of *features* it unblocks. The 1+ avoids cancelling the product for a *feature* with no successor. The lower the score, the stronger a candidate for removal the *feature* is: low business value and few successors blocked behind it. The cascade applies `on_delete=CASCADE` at the Feature→Story level and flags the potentially orphaned DAG successors.

6.11 Capacity bottleneck detection

$$demand(s) = \sum_{t: skill(t)=s} d_t \quad (15)$$

$$supply(s) = \sum_{r: s \in skills(r)} avail_r \cdot PI_{days} \quad (16)$$

$$gap(s) = \max(0, demand(s) - supply(s)) \quad (17)$$

7 Design decisions

Table 2 synthesizes the major design decisions with their justification. Two sources have fed these choices: the recent scientific literature on the one hand, and on the other a **semi-structured interview with a Product Owner** operating on a SAFe ART. This interview does not have the value of an exhaustive field study, but it made it possible to frame elements that no article documents precisely, the PI format of 4 × 3 weeks, the empirical story points/days conversion, or the strict application of the 20 % buffer. When a decision relies on this interview, the mention *PO Interview* indicates it in the table.

A **Status** column states what each decision is worth today, and it should be read without indulgence:

- **Measured:** the experimental campaign (section 9) shows, with figures, that the decision improves the outcome;
- **Checked:** an automated test guarantees that the system behaves as the decision intends, without saying whether it was the right choice;
- **Assumed:** the choice rests on the literature or on the interview, but nothing in this thesis demonstrates it. These cases are picked up and discussed in section 9.9.

Ref.	Decision	Justification (academic sources + field observation)	Status
DEC-001	Team specialization (enum + <i>fullstack</i> fallback)	[21]: an ART contains 5-15 typed teams. <i>PO Interview</i> : real ARTs are organized into <i>back</i> / <i>front</i> / <i>data</i> / <i>devops</i> / <i>mobile</i> / <i>qa</i> teams with a cross-cutting fullstack team for cross topics.	Assumed
DEC-002	<code>span_factor</code> pivot 2.0, drawn in [1.8; 2.2] per team	[25] Kingman’s law ($\approx 50\%$ utilization $\Rightarrow$ lead-time $\approx 2\times$).	Assumed
DEC-004	Global exploratory then refinement	[3] (two-phase memetic); [12] (two-stage HGA).	Measured
DEC-005	Warm re-planning of team counter-proposals	[12]: two-stage scheme. Validates the <i>feature</i> transfers proposed by the teams before the convergence of the <i>swarm</i> .	Assumed
DEC-006	feature→team cohesion: penalty $\times 100$	[21]: “one team owns one feature”; [15]: flexible profiles with cost. <i>PO Interview</i> : splitting a feature across several teams is treated as a negotiated exception, never as a default case.	Checked
DEC-007	Pareto diversity via farthest-first Hamming	[6]: multi-agent crowding; [8]: congestion operators.	Assumed
DEC-008	Runtime <code>DecisionTrace</code> model	[7]: review insisting on combinatorial traceability.	Checked
DEC-010	PI 12 weeks $\times$ 4 sprints of 3 weeks	[21] (PI 8-12 wk.); <i>PO interview</i> : 4×3 weeks format actually used (consistent with the quarterly <i>cadence</i> of Business Owners).	Assumed
DEC-014	AI advice + HITL rebalancing at step 3	[22]: ML/MOEA integration for decision support; [21]: ROAM board. <i>PO Interview</i> : RTEs perform an explicit <i>capacity review</i> and negotiate removals feature by feature.	Measured
DEC-015	Explicit RTE scenario choice at step 5	[4]: max-regret for multi-actor decision; [21]: mandatory Business Owner review. <i>PO Interview</i> : the final decision is never delegated to a mechanical consensus, the RTE decides in the presence of the Product Owners.	Checked
DEC-016	Blocking 80 % buffer	[25] (Kingman); [21]. <i>PO Interview</i> : the 20 % buffer rule (capacity load $\leq 80\%$) is strictly applied by RTEs before PI validation.	Measured
DEC-018	Safety margin per <i>story</i> , inspired by the Critical Chain	[24]; taken up in [17]; [22]: ML+MOEA.	Assumed
C1	WSJF-weighted tardiness (6 th obj.)	[14]: multi-project industrial case, <i>weighted tardiness</i> .	Assumed
C2	WSJF/deadline urgency seed	[1] INSGA-II; [2] improved variant on an industrial case.	Assumed
C3	Memetic local search	[3]: <i>greedy</i> local search on front 0.	Measured

Table 2: Major design decisions, with their academic sources and, for the choices concerned, the elements from the PO interview.

8 Flow logic problems and solutions

The flow described in section 5 was not designed in a single pass. Its first version was more naive, and several logic flaws only appeared when running the system end to end on the reference dataset: a consensus forming on misleading figures, a step traversed without noticing it, an optimizer producing an absurd but “valid” plan. This section gathers these flaws and the correction I made to each. These are not code bugs but errors of **flow design**; I present them under the form *symptom* then *correction*, referring to the corresponding DEC-XXX decision.

8.1 Team agents negotiating on SP aggregates

Symptom. The first version of the flow had the team agents negotiate on estimates aggregated in *story points*. Consequence: their arguments were vague, their portfolio refusals inexplicable, and the negotiation between teams could converge on a consensus infeasible on the resource and calendar side, an internal bottleneck (for example a senior resource overloaded at 130 % on sprint 2) remained hidden behind the SP average.

Correction (DEC-004). Aligned on the two-phase approach of the Memetic NSGA-II [3]: one first

runs a global exploratory pass that produces concrete schedules (*story*→resource assignment, dates, load per sprint), then these schedules are given to the team agents so that they negotiate on real data. A *warm-start* re-scheduling validates the counter-proposals before convergence (DEC-005), in line with the two-stage scheme of [12].

8.2 NSGA-II splitting features across several teams

Symptom. Without an explicit constraint, the optimizer produced technically excellent plans but distributing the *stories* of the same *feature* across 3-4 different teams, a flagrant violation of the SAFe principle “one team owns one feature” [21]. The real process admits this breakdown (an epic is sometimes distributed into features across several teams, section 2.1) but treats it as a negotiated exception, never as a default case.

Correction (DEC-006). Splitting is not forbidden, it is heavily penalised: each *story* placed outside the majority team of its *feature* multiplies the corresponding contribution to the score by 100. The optimiser therefore avoids it spontaneously, while retaining the

possibility of resorting to it when the split is legitimate (*component teams* [21]). A hard constraint would have made the problem infeasible on those edge cases.

8.3 SAFe 20 % buffer not respected at step 3

Symptom. The empirical SAFe rule (derived from Kingman's law [25]: $>80\%$ utilization $\Rightarrow$ super-linear lead-time) is that one must not exceed 80 % of planned load. Yet step 3 let the flow pass to NSGA-II as long as the load remained $\leq 110\%$. NSGA-II then started on an infeasible portfolio and found the "least bad" compromises instead of arbitrating upstream.

Correction (DEC-016). Blocking threshold lowered to 80 % by default: as long as at least one team exceeds this threshold or refuses its portfolio, the flow remains in HITL at step 3. The rule is spelled out in a banner citing Reinertsen [25].

Field nuance. The PO interview confirmed that in practice **each team defines its own flow**, and that the percentage of capacity actually dedicatable to PI Planning varies according to the maturity, the topology and the **external load** of the team (support volume, influx of *Problem Change Requests* following an acceptance-testing campaign, etc.). The 80 % of the SAFe buffer is therefore a *default target value* and not a universal constant. The blocking threshold is configurable per team in the data model; it is exposed to the RTE so that he can lower this threshold on the teams known to absorb a lot of work outside the PI.

8.4 Overload without a rebalancing loop

Symptom. When step 3 detected an overload, the flow offered **no automatic rebalancing mechanism**: the only possible exit was to continue toward NSGA-II with an infeasible allocation. The algorithm therefore started on an intrinsically oversized portfolio and produced compromises suboptimal by construction.

Correction (DEC-014). Introduction of a **deterministic AI advice** (equation 14) that orders the candidate *features* by increasing impact, and of a **re-CNP loop** (Capacity Negotiation Protocol) bounded by the 80 % rule. Following the spirit of the *survey* [22] on ML/MOEA integration in multi-objective decision support, the loop only closes once the capacity constraint is satisfied. The optimizer then receives a healthy portfolio rather than having to compensate downstream.

8.5 Step-3 removals ignored by step 4

Symptom. The rebalancing loop above did produce a list of removed *features*, but step 4 never received it: the optimiser reloaded the ART's entire *backlog* from the database. The RTE therefore saw *features* he had just explicitly taken out of the PI reappear in the final schedule. The symptom was all the more insidious in that no error was raised and the resulting schedule looked perfectly valid.

Correction. The excluded identifiers are passed down to the loading of the optimisation context, which restricts the scope accordingly, and the schedule can no longer contain a *story* belonging to a removed *feature*. The property is locked down by an automated end-to-end test: without it, the regression would have gone unnoticed, since it shows up neither as an error

nor as a visibly inconsistent schedule.

8.6 Auto-finalization short-circuiting the final decision

Symptom. Step 5 auto-finalized on consensus as soon as the team agents agreed on two preferential strategies or fewer. The *flow* then jumped directly from step 3 to a state "PI Planning finished" without having materialized an explicit choice: the scenario "recommended" by the system was considered accepted by default.

Correction (DEC-015). Step 5 made **always** HITL by construction of the flow. Inspired by the *max-regret* of [4], the orchestrator computes an indicative consensus (majority of votes, recommended by the *composite score*, Pareto-optimal) but never triggers the transition toward step 6 without an explicit decision. This respects the semantics of the conditional HITL: conditional *on an algorithmically detectable problem*, and not on a fundamentally human business decision [21].

Alignment on the real course of a PI Planning. The ritual observed at the interviewee's takes place over two days with a clear structure, which we seek to reproduce in the tool:

- **Day 1.** Each PO presents the proposal of his team; the stakeholders (RTE, CPO, EDM, dependent teams) accept or refuse. A *synthesis sharing* at the end of D1 makes it possible to identify the difficulties to anticipate for D2.
- **Day 2.** The teams refine their commitments by integrating the previous day's counter-proposals. A *final validation review* at the end of D2 enacts the definitive plan.
- **Possible outcome.** It happens that the plan *is not accepted* at the end of the two days: in this case, other meetings are held during the first days of the PI itself to bring the teams and the stakeholders toward a validated plan.

SAFESCALE models this structure in two phases on the front-end side: the K scenarios are first presented in *review* mode (analogous to D1, each team votes, the hard points are identified), then the RTE performs the *final validation* (D2) from the votes and the evaluator's advice. The absence of explicit validation at the end of D2 produces a state "PI Planning awaiting arbitration" rather than a state "finished".

8.7 Pareto front giving the illusion of choice

Symptom. The Pareto front contained a few dozen solutions, many of which were neighbors of the same optimum. Selecting $K = 5$ by composite score produced scenarios visually different but qualitatively identical.

Correction (DEC-007). We force the **diversification** of the scenarios proposed to the RTE: instead of taking the $K = 5$ best by composite score, we build the list one by one by taking each time the scenario most different from those already chosen. The distance used is the *Hamming distance* on the feature $\rightarrow$ team assignment (cf. section 6 for the detail). Following [6] (front uniformity) and [8] (congestion operators).

8.8 Execution variance and under-activity teams

Field observation. The major constraint observed today on ARTs remains **over-capacity**: teams committed beyond what they can absorb. But the PO feedback also describes the reverse phenomenon: teams **under-activity** on a PI, either because they have a higher velocity than expected, or because their portfolio deflated between planning and execution.

Symptom on the optimizer side. With equation 2 alone, two problems appear:

- a team planned at 80 % sees its schedule compact over 2 sprints whereas it has 4, without any margin for execution variance;
- the optimizer proposes nothing to make use of the capacity freed up by an under-activity team.

Correction (DEC-018), for execution variance. Insertion of **safety margins** targeted at the high-variance *stories* (equation 7), following Goldratt's principle [24] and the ML+MOEA *survey* [22]. A variant of the *fuzzy RCPSP* [17] that converts uncertainty into objectives; here we convert it into local buffers.

Lead for under-activity (to explore). When a team is under-utilized, two business levers are conceivable, in addition to the buffers:

- **Anticipate stories from the next PI:** pull up in the backlog features planned for PI N+1 and add them opportunistically if the skill fits;
- **Take on stories from other teams** that are overloaded, provided the specialization allows it (*cross-team support*, typically by a *fullstack* team).

These two movements are already partially modelable in SAFESCALE: the first fits into the feature removal/addition mechanism of step 3, the second depends on the specialization model already described in section 4. Their explicit integration is listed as perspective P7 (section 12).

8.9 Uninstrumented per-skill capacity bottleneck

Symptom. A bottleneck on a rare skill (*ml_ops*, *mobile*, *devops*) never stands out as an isolated signal: the overload manifests at the team scale and drowns the origin. The RTE removes *features* "at random" without knowing which one unblocks the bottleneck.

Correction. A *skill_bottleneck* service that computes *demand*, *supply* and *gap per skill*, identifies the covering teams, and weights the candidate *features* for removal by the effort they free up on the bottleneck skill. Extreme case: a skill not covered by any team $\Rightarrow$ bonus +5 to the removal score of the *features* that require it. This modeling relies on the notion of *flexible resource profile* of [15].

9 Tests and results

The preceding sections describe a system and justify its choices. This one asks what it is actually worth. The difference matters: so far, everything stated rested on the literature, on the interview with the Product Owner, or on common sense. Nothing had been measured.

9.1 The four questions

A decision-support tool can miss its target in several ways, and they are independent of one another. It can propose schedules that cannot be executed. It can propose executable ones that are no better than what one would get by sorting the work by hand. Or it can rest on algorithmic refinements that, once measured, change nothing. Hence four questions, asked in the order in which a negative answer would make the following ones moot:

- **Q1. Are the proposed schedules workable?** A *story* cannot start before the one it depends on, one person cannot work on two *stories* the same day, and nothing must have been left out along the way.
- **Q2. Do the two improvements really help?** These are the starting schedules built from business rules (C2) and the touch-up of the best schedules at the end (C3).
- **Q3. Does the system beat a simple method?** We compare it against sorting the *stories* by priority, then look at how much is left to gain before hitting the physical limit of the problem.
- **Q4. Does the 80 % rule change the outcome?** In other words, does the block at step 3 serve any purpose, or is it just a formality?

9.2 How the campaign is run

The datasets. The campaign covers 5 datasets, produced by the generator with different seeds. One would not be enough: we would then describe the quirks of a single draw rather than the behaviour of the algorithm. The seed 42 dataset serves as the reference when going into detail.

Dataset	Stories	SP	Cap.	Ratio	Prec.	Pool	LB
42	112	801	625	1.28	207	4.0	281
43	131	929	710	1.31	246	3.7	183
44	131	952	640	1.49	223	3.7	204
45	129	892	645	1.38	436	3.9	169
46	129	971	610	1.59	281	3.8	224

Ratio: what is asked of the teams divided by what they can absorb. Above 1, the PI is overloaded. **Prec.:** number of "this one before that one" constraints between *stories*; when two *features* follow each other, all their *stories* inherit the constraint, hence the high count. **Pool:** how many people can pick up a *story*, on average; the lower it is, the less choice there is. **LB:** the number of days below which no schedule can go.

Two columns deserve a word. The **ratio** exceeds 1 everywhere: the generator deliberately creates an overloaded PI, since that is precisely the situation PI Planning has to arbitrate. And the number of **precedences** is far higher than one would guess from counting the dependencies that were entered: when one *feature* must wait for another, it is all the *stories* of the first that must wait for all those of the second. Counting only the entered dependencies would suggest a problem with almost no ordering constraints.

The three versions compared. We compare three versions of the same algorithm, each adding one brick to the previous one:

- *standard:* NSGA-II as described in the literature, starting from randomly drawn schedules;

- *+seeds*: the same, but starting from the six schedules built by business rules (section 6);
- *+memetic*: the previous one, plus the touch-up of the best schedules at the end.

All three get the same computation budget: 60 schedules carried in parallel, 120 generations. Giving them different budgets would measure computation time rather than the contribution of the added brick.

Why repeat. The algorithm involves a share of randomness: two identical runs do not give the same result. Each version is therefore launched 10 times on each dataset, i.e. 50 measurements per version.

What is measured. The system does not propose one schedule but a batch of them, each a different trade-off between the six objectives. Judging a batch therefore takes several indicators, because none of them answers every question on its own. The technical names are given in brackets, for the reader who would want to look them up.

- the **coverage**, called *hypervolume* in the literature: how much of the range of possible trade-offs does the batch actually occupy? The more it covers, the more choice the RTE has.
- the **distance to the best** (IGD+ [28]): how far is the batch from the best we obtained, across all versions and all runs? That “best” is the result of no single run: it is assembled afterwards, by gathering what each run produced at its best and keeping only what nothing surpasses. No version ever reaches it entirely; it serves as a common yardstick.
- the **evenness** (*spread*): are the schedules evenly spaced, or clustered into a few groups separated by gaps? An RTE choosing between three near-identical variants does not really have a choice.
- the **overlap rate** (C-metric [27]): comparing two batches schedule by schedule, does one systematically do at least as well as the other?

One methodological point conditions everything that follows. Coverage is always computed relative to a reference area, and it means nothing if that area changes from one batch to the next: a wider batch can then score lower, which inverts the ranking. The first version of the code did exactly that. The reference area is therefore now computed once per dataset, over everything the three versions produced, and all batches are measured against it.

Deciding whether a gap matters. Comparing two means is not enough. Since the algorithm involves a share of randomness, a higher mean may be nothing but a run of lucky draws. Each comparison therefore comes with two figures, answering two distinct questions.

The first is the **risk that it is chance**. We ask: if the two versions were really equivalent, what would be the probability of seeing a gap at least as large as the observed one? When that probability falls below 5 %, we rule chance out.

The second is the **magnitude of the gap**. We pit every run of one version against every run of the other,

and count the share of duels the first one wins. At 0.5, the two versions are equivalent: each wins half the time. At 0.8, the first wins four duels out of five.

Both figures are needed, for opposite reasons. The first alone says nothing about importance: by multiplying runs, one always ends up making a minuscule gap look “unlikely”. The second alone says nothing about reliability: a gap may look wide and be nothing but a draw accident.

For the reader who wants to check: the first figure is the p -value of a Mann-Whitney test, the second the A_{12} statistic of Vargha-Delaney [29, 30]. Probabilities are adjusted with Holm-Bonferroni, because the three comparisons rest on the same data and multiplying tests mechanically raises the risk of one passing by chance.

Replaying the campaign. Everything re-runs with a single command (`manage.py run_experiments`). It works in a temporary database, created then dropped, and writes both the raw results and the tables of this section. No figure is therefore copied by hand: re-running the campaign regenerates them, and the slightest divergence shows up immediately. Count about 62 minutes.

9.3 Q1. Are the proposed schedules workable?

A batch of schedules that cannot be executed is worthless, and the algorithm would not flag it: it optimises what it is asked to measure, not what one forgot to check. Every proposed schedule is therefore taken back and checked on four points: a *story* never starts before the ones it depends on are finished, safety margin included (the one inserted after the riskiest stories, section 6.4); nobody works on two *stories* at the same time; each *story* is carried out by someone from the team that owns it; and none has been left out.

Across the whole campaign, 100.0 % of the schedules respect these four points. This is not luck: it follows directly from the way a schedule is built. The program places each *story* as early as possible, once its predecessors are done and its resource is free; and the operations that make schedules evolve from one generation to the next refuse any reordering that would break the dependency order. This is checked separately at start-up and after each kind of modification: an overall rate of 100 % could otherwise hide an operation that is never triggered.

9.4 Q2. Do the two improvements really help?

The question is a simple one. NSGA-II is a published, well-tried algorithm; we added two things to it. First **starting schedules** built from business rules (C2), instead of starting from randomly drawn ones. Then a **final touch-up** (C3), which picks up the best schedules of the batch and searches around them. Each represents real work: code to write, to test, to maintain. Do they help?

To answer, we compare the three versions on the indicators defined above. The first table gives the mean values.

Version	Coverage	Dist. to best	Evenness	Unranked	Workable	Time (s)
Standard NSGA-II	0.417	0.090	0.777	100.0 %	100.0 %	6.1
+ seeds (C2)	0.409	0.108	0.761	100.0 %	100.0 %	6.1
+ memetic (C3)	0.455	0.081	0.774	100.0 %	100.0 %	7.1

5 datasets $\times$ 10 runs each, i.e. 50 measurements per version. **Coverage**: how much of the range of possible trade-offs the batch actually occupies; the higher, the more choice the RTE gets. **Dist. to best**: how far the batch sits from the best seen across all versions; lower is better. **Evenness**: whether the schedules are evenly spaced or clustered with gaps; lower is better. **Unranked**: share of the 60 schedules in the batch that no other one beats; at 100%, NSGA-II's sorting criterion no longer sorts anything. **Workable**: share of the schedules that can actually be executed.

One row stands out and another disappoints. The *+memetic* version achieves the best coverage (0.455) and the smallest distance to the best (0.081). The *+seeds* version, on the other hand, does no better than the standard: its coverage is even slightly lower (0.409 against 0.417).

Whether these gaps are real or down to the luck of the draw is the second table's business.

Comparison	A	B	Chance risk	Magnitude	Effect
+ seeds (C2) vs Standard NSGA-II	0.409	0.417	0.9067	0.507	not significant
+ memetic (C3) vs Standard NSGA-II	0.455	0.417	0.0065	0.678	medium
+ memetic (C3) vs + seeds (C2)	0.455	0.409	0.0091	0.665	medium

A and **B**: mean coverage of the two versions compared. **Chance risk**: probability of seeing such a gap when the two versions are in fact equivalent; below 0.05 we rule chance out. **Magnitude**: 0.5 means the two versions are equivalent, and the further from it, the more the gap weighs. Both are needed: a minuscule gap always ends up looking "unlikely" if enough runs are made. (Mann-Whitney test, probabilities adjusted with Holm-Bonferroni for the three comparisons.)

The verdict is clear both ways.

The starting schedules (C2) do not improve the final result. The gap with the standard version is not significant: the risk that it is chance stands at 0.9067, far above the 5% threshold, and the magnitude at 0.507, i.e. roughly one duel won out of two. In other words, starting from six schedules built by business rules does not give a better batch at the finish.

The explanation lies in variety. Those six schedules are all *good* and all *alike*. They weigh only six schedules out of 60, but since they are better than the others, they are kept first from the very first generations, and their assignments reappear in the schedules produced by crossover, generation after generation, until the whole population is pulled into the same area. This is the usual dilemma of such algorithms: what you gain by starting well, you lose by exploring less. Over 120 generations, the two effects cancel out.

Dataset	Standard NSGA-II	+ seeds (C2)	+ memetic (C3)
42	0.534	0.485	0.532
43	0.432	0.439	0.489
44	0.386	0.386	0.435
45	0.362	0.425	0.431
46	0.372	0.307	0.390

Mean coverage over 10 runs, dataset by dataset; the best of each row in bold. This breakdown shows what the mean hides: a configuration that wins everywhere is not worth the same as one that wins here and loses there.

The dataset-by-dataset breakdown shows that "no gap on average" does not mean "no effect". The starting schedules do clearly better than the standard on dataset 45 (0.425 against 0.362) and clearly worse on dataset 46 (0.307 against 0.372). The effect therefore exists, but it flips with the shape of the *backlog*, and the mean only cancels it because the good and the bad cases compensate. Had we concluded on a single

dataset, we could have presented C2 as a gain or as a loss, depending on which one we picked.

The final touch-up (C3) does bring a real gain. Its coverage reaches 0.455 against 0.417 for the standard, with a chance risk of 0.0065 and a magnitude of 0.678, i.e. about two duels won out of three. The gap with *+seeds* is of the same order (0.0091 and 0.665). Above all, the advantage is **stable**: this version wins on four datasets out of five, and trails by one thousandth on the fifth. The cost is modest: 7.1s per run against 6.1s.

Figure 3 shows where that gain comes from. It tracks the coverage of the batch across generations, averaged over the five datasets. Three things can be read from it.

First, the two versions starting from constructed schedules really do begin ahead: from the very first generation they cover 0.225 against 0.196 for the standard. The work invested in the business rules is therefore not without effect, it is simply early.

Second, the starting-schedules curve plateaus. The standard catches up around the fiftieth generation and then passes it: this is the loss of variety described above, seen directly.

Third, the memetic version keeps its lead from start to finish and even widens it. At each pass, the touch-up picks up the most isolated schedules of the batch and goes looking for trade-offs the other two versions never find.

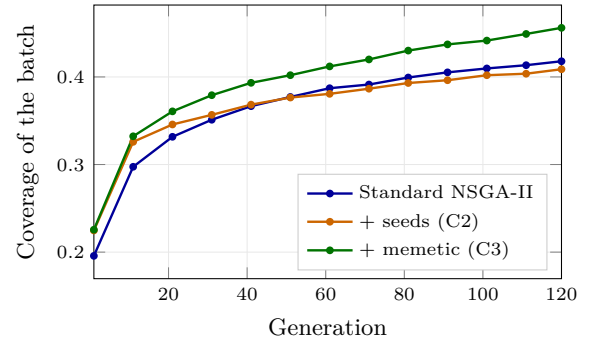


Figure 3: Coverage of the batch of schedules across generations, averaged over the 5 datasets and the 10 runs of each. This is the same indicator as the first column of the results table, followed step by step: the last-generation values are the table's. The starting schedules begin ahead then get overtaken; the final touch-up keeps its lead.

$A \setminus B$	Standard NSGA-II	+ seeds (C2)	+ memetic (C3)
Standard NSGA-II	—	0.360	0.243
+ seeds (C2)	0.107	—	0.115
+ memetic (C3)	0.194	0.358	—

Measured on the reference dataset. $C(A, B)$ = share of B 's schedules for which A offers at least as good. The two directions are read together: if A covers much of B and B little of A , then A is better; if both values are close, the two simply explore different trade-offs.

A last table compares the batches two by two, on the reference dataset, and confirms the previous reading by another route. Between *standard* and *+memetic*,

neither takes the lead: the first does at least as well on 24.3 % of the second's schedules, which returns the favour on 19.4 %. They simply offer different trade-offs, and the memetic version's advantage comes from the extent of what it offers, not from a schedule-by-schedule superiority. On that dataset, by contrast, *+seeds* is beaten both ways: the two other versions do at least as well on 36.0 % and 35.8 % of its schedules, while it only manages 10.7 % and 11.5 % of theirs.

With six objectives, the algorithm can no longer rank. In all three versions, 100 % of the final population is judged "unbeaten": no schedule is better than another on all six criteria at once. This is not good news, but it is a known phenomenon when objectives pile up [9, 11]: the more there are, the rarer it becomes for one schedule to beat another everywhere, and the sorting criterion ends up sorting nothing. The algorithm then falls back entirely on *crowding distance*, a mechanism meant to preserve variety, not to rank.

This also sheds light on the previous result: if the sorting no longer does its job, starting well matters less than exploring well, which is what the final touch-up provides and what the starting schedules do not. This is the clearest quantitative argument for moving to NSGA-III (section 12, P2), which replaces that sorting criterion with another one, designed to stay useful beyond three or four objectives.

9.5 Q2 bis. What if we allowed less computation time?

The previous result is awkward: the starting schedules require real work (six business heuristics written, tested, maintained) and the measurement finds them no effect. Before declaring them useless, one hypothesis remains to be ruled out, and figure 3 points at it: the lead of the starting schedules is real, but it only lasts for the first few generations. If everything hinges on that, the lead should still be visible when the algorithm is stopped early, before the standard has had time to close it.

The question is not theoretical. During the discussion, the application does not let the optimisation run for 120 generations: the RTE is waiting for an answer, and step 4 stops at 60. That is therefore the budget at which C2 must be judged, not the comfortable budget of the main campaign. We thus re-run the three versions at four increasing budgets.

Generations	Standard NSGA-II	+ seeds (C2)	+ memetic (C3)	Seeds vs standard
20	0.336	0.352	0.365	$p = 0.1234$, $A_{12} = 0.59$ (not significant)
40	0.372	0.375	0.400	$p = 0.5304$, $A_{12} = 0.54$ (not significant)
60	0.394	0.389	0.420	$p = 0.8093$, $A_{12} = 0.51$ (not significant)
120	0.426	0.419	0.466	$p = 0.8254$, $A_{12} = 0.51$ (not significant)

Mean coverage over 10 runs and 5 datasets, at increasing computation budgets. The last column compares the starting schedules (C2) against the standard version at the same budget. The application stops at 60 generations during the discussion: that row is the one deciding whether C2 helps in real conditions.

The sweep does not save C2. At 20 generations the gap does point the expected way (0.352 against 0.336, $A_{12} = 0.59$), but it remains not significant ($p = 0.1234$): the trend exists, it is not established. And at 60 generations, the budget the application actually uses, nothing is left of it at all ($A_{12} = 0.51$, the equivalent of a coin toss).

The hypothesis was therefore half right. A trace of an advantage does survive at the tightest budget, but it is too faint to be told apart from chance, and it has vanished before even reaching the application's 60 generations. We found no regime in which C2 justifies itself by the quality of the batch produced. The choice is thus reclassified *assumed* in table 2, and picked up in section 9.9: it is not condemned, it is simply not demonstrated.

The table does bring a useful confirmation about C3, though: the final touch-up leads at all four budgets, including the tightest. Its value therefore does not depend on how much time it is given, which was not a foregone conclusion for a refinement technique: one could have feared it would not have time to act.

9.6 Q3. Do we beat a simple method?

Comparing the three versions with each other says nothing about whether the whole thing beats a simple method. We therefore pit the system against four elementary ways of planning: sorting the *stories* by business value, by deadline, by length of dependency chain, or by balancing the load, then placing them in that order. These four methods are evaluated on the same dataset, **with exactly the same schedule computation and the same objective formulas**: the observed gap cannot come from a difference in measurement, only from the way of searching.

What counts as a good result still has to be established. The best possible duration is unknown: that is precisely what makes the problem hard. We therefore compute three durations below which no schedule can go, and keep the tightest: the longest chain of *stories* that must follow one another, a team's workload spread as evenly as possible among its members, and above all the case where a single person can handle a whole set of *stories*. They then run them one after another, and that duration is incompressible.

Method	Duration (d)	Capacity over. (SP)	Weighted delay
Greedy WSJF	282	307	89.2
Greedy deadline	282	307	89.6
Greedy critical path	282	295	74.5
Greedy load balancing	282	307	89.2
Standard NSGA-II	281	50	70.8
+ seeds (C2)	281	54	67.4
+ memetic (C3)	281	55	66.6
Lower bound	281	—	—

Reference dataset (seed 42). A simple method yields a single schedule, NSGA-II yields several: for it we keep the best of its batch in each column. The lower bound is the tightest of three incompressible durations: the longest chain of dependencies (101 d), the person who alone can handle a set of *stories* (281 d), and a team's workload spread as evenly as possible (114 d).

The table reads in two steps.

On duration, there is nothing to gain. The system finishes in 281 days, the simple methods in 282, and the limit computed above is 281 days: the system is therefore exactly on that limit, and the simple methods one day away from it. This is not an algorithmic failure, it is a characteristic of the dataset: 19 *stories* can only be taken by one person, and the one concentrating the most of them has to run them one after another, i.e. 281 days on their own, far more than the

longest dependency chain (101 days) or a team's workload (114 days). No schedule can do better. And that is a useful result in itself: it says that **to shorten this PI, one has to train or hire on the rare skills, not improve the algorithm.**

That perfect match is specific to the reference dataset, and it would be dishonest to turn it into a general rule. Across the 5 datasets, the gap to the limit ranges from 0 % to 63 %, averaging 24 %. Beware of reading that gap as the system's room for improvement, though: this limit is a guarantee of the "one cannot do less" kind, not the best duration actually reachable. A large gap may equally well mean that the limit is too low. The two datasets where it matches the result (0 and 1 % gap) are precisely those where a single person, or a whole team, saturates; elsewhere, none of the three limits describes what is really blocking.

On the rest, the gap is considerable. Team capacity overrun falls from 295 SP for the best simple method to 50 SP, i.e. 5.9 times less; and the delay on high-business-value *features* recedes by 10.5 %. That is where the system's real contribution lies on this dataset: not finishing earlier, which is impossible, but spreading the load without blowing up the teams and delivering first what matters most.

9.7 Q4. Does the 80 % rule change the outcome?

Section 8 defended the block at step 3 with a theoretical argument: beyond 80 % load, delays grow much faster than the load itself [25]. The experiment measures what that rule changes in practice.

A small program replays the arbitration expected from the RTE: as long as a team exceeds 80 % load, it removes the least useful *feature* in the sense of equation 14: low business value and few *features* blocked behind it. What remains is then re-planned, and compared.

Dataset	Features rm.	Max load	Duration (d)	Capacity over. (SP)
42	10	1.60 → 0.79	281 → 128	55 → 3
43	11	1.75 → 0.73	255 → 115	27 → 0
44	13	1.91 → 0.77	206 → 85	76 → 2
45	12	2.32 → 0.77	275 → 147	58 → 2
46	12	1.86 → 0.75	267 → 88	91 → 2

Left of the arrow, the PI as it reaches step 3; right of it, the same PI after removing the least useful *features* one by one, until no team exceeds 80 % load.

The effect is large and points the same way on all five datasets. On the reference one, 10 *features* leave the PI, the load of the busiest team falls from 1.60 to 0.79, the duration from 281 to 128 days and the capacity overrun from 55 to 3 SP.

Two conclusions, one of them uncomfortable. First, the rule does serve a purpose: without it, step 4 returns a schedule that *looks* complete, since every *feature* is in it, but which would take more than four times the PI duration. The block is what stops the system from having the ART commit to something undeliverable. Second, the scale of the removal (521 SP out of 801, close to two thirds of the *backlog*) mostly reflects a deliberately overloaded dataset; it is not a prediction of what an RTE would do. In a real PI Planning, *features* are also split, deferred to the next PI, or renegotiated:

three levers the program knows nothing about. What the experiment establishes is therefore the **direction** of the effect and its order of magnitude, not the exact number of *features* to remove.

9.8 What these results do not prove

The results above hold within a precise scope. Better to say so than to leave it to be guessed.

The data is fabricated. It is dimensioned after an interview with a Product Owner (section 4), but a generator reproduces what it was told, not the irregularities of the field. Nothing guarantees that the conclusions hold as-is on a real ART, in particular for the dependencies, more regular here than on a real *Program Board*.

Risk is computed by a rule, not learned. The machine learning mode exists, but it could not be trained for lack of real historical data. The "risk" objective therefore measures a rule we wrote, not a regularity observed in the field.

Absolute durations are approximate. They rest on the span factor (equation 2), itself an estimate. If it is somewhat off, all durations shift in the same proportion: comparisons between versions remain valid, but the day counts should be taken as orders of magnitude.

Some gaps remain undecidable. The statistical tests rest on 50 measurements per version. That is enough to detect a clear gap, not enough to settle a small one. When a gap is declared "not significant", it means "not demonstrated here", and certainly not "demonstrated equivalent".

What could not be tested. The **system versus human PO** comparison, announced as a second protocol, did not take place: it requires a PO to be available for a full PI Planning session, which was not possible. This is the main gap in the evaluation, since it alone would say whether the system is genuinely useful to those who use it, and not merely correct. The other announced protocol, the name automatically given to each scenario, was on the other hand implemented and is checked by the end-to-end tests (section 10).

9.9 What remains assumed without proof

The **Status** column of table 2 distinguishes three cases: *measured*, *checked* and *assumed*. That last category deserves to be picked up here, because it is the most fragile and the easiest to pass over in silence.

An *assumed* choice is not an arbitrary one: it rests on the literature or on the interview with the Product Owner. But nothing in this thesis shows that it produces a better outcome than some other option. Five deserve to be flagged.

- **The span factor** (DEC-002). Its pivot value of 2.0 comes from Kingman's law at 50 % utilisation. No real data confirmed it here, and it conditions every duration reported.

- **The selection of the five scenarios shown to the RTE** (DEC-007). We pick the ones furthest apart from each other, to avoid offering five near-identical variants. That is reasonable, but we did not compare that rule with other ways of choosing, nor ask an RTE which one helps most.
- **The safety margin per story** (DEC-018). The principle comes from Goldratt's *Critical Chain*, but the formula sizing it, like the weights of the five factors it combines, were set by us.
- **The weights of the composite score** (DEC-015). They designate the recommended scenario by default. We set them by hand, without comparing them to other weight vectors nor asking an RTE which one matches his arbitration.
- **WSJF-weighted tardiness** (C1). This sixth objective is taken from an industrial case [14]. We did not measure what the system would give without it, and section 9.4 shows precisely that adding objectives has a cost: beyond four or five, NSGA-II's sorting criterion stops working.

To these five cases a sixth is added, and it is the most instructive, because we tried to get it out of the list. **The starting schedules** (C2) were the subject of a dedicated experiment (section 9.5), set up precisely to check whether they helped on a tight budget. The answer is no, or more exactly: not demonstrated, at any of the four budgets tested.

We keep them nonetheless, and it is better to say why than to let it look like an oversight. Three reasons. They cost nothing: computation time is identical to the standard version. Their early lead is real, figure 3 shows it; it simply does not survive to the end. And the dataset-by-dataset breakdown shows they sometimes do clearly better, on one of the five datasets, without our being able to predict on which shape of *backlog* that happens. An occasional, free gain is still preferable to no gain at all. But it is a bet on usage, not a measured result, and it must be read as such.

Stating this list is not an admission of weakness but a methodological precaution: it says exactly where to put the effort if the work were to be carried on, and it prevents the whole thesis from being read as if everything in it had been demonstrated.

10 Feedback on the solution

10.1 Qualitative observations

The feedback that follows is **qualitative**: it comes from demonstration sessions conducted internally on the reference dataset. It complements the quantified evaluation of section 9 without replacing it: what is described here concerns the use of the system, not the quality of the schedules it produces.

Flow legibility. Breaking the process into five named steps (*Preparation, Dependencies, Capacity & allocation, Scheduling, Scenario choice*) makes each arbitration easier to follow. One can resume a session step by step and explain what happened, which is useful for the defense.

Effect of the 80 % buffer. As long as the blocking threshold did not exist, step 3 was traversed quickly

and without real discussion. By making it blocking, it becomes a moment of negotiation: the RTE sees explicitly why he must remove *features*, since the rule is displayed.

Removal advice. The removal panel at step 3 orders the candidate *features* by impact score (equation 14). In practice, it highlights unintuitive arbitrations, for example that a *feature* with average WSJF without a *deadline* frees up more margin than a *feature* with high WSJF but a tight *deadline*.

10.2 Pareto consistency

The *farthest-first* selection (DEC-007) produces visibly distinct scenarios: one can give them speaking labels ("fast", "safe", "lean"). This is the intended effect, avoiding presenting to the RTE five variants of the same compromise.

How these labels are generated. Once the five scenarios are selected, we look at which of the six objectives each is the **best of the lot** on and derive its name:

Best on...	Generated label
makespan	"fast"
risk score	"safe"
capacity violation	"lean"
WSJF tardiness	"priority"
buffer deficit	"with margin"
overload	"balanced"

The names are deliberately **short and business-oriented**: the PO and the RTE can use them to talk to each other ("I prefer the fast scenario, but with a bit more margin") without knowing the mathematical formula that determined the ranking. If several scenarios are best on the same objective, we decide by the second best KPI, which produces compound labels ("fast-safe").

10.3 Reproducibility

The fixed *seed*, the LLM *mock* mode and the **DecisionTrace** trace make it possible to replay a session identically. This is important here: without it, one could neither compare two versions of the system, nor verify a result during the defense.

A fourth, less obvious condition turned out to be necessary: the loading queries must impose an **explicit ordering**. A fixed seed is of no help if the database returns the same rows in a different order from one run to the next. Two campaigns launched with the same settings otherwise produced different figures.

11 Limits and what is not taken into account

The scope of the prototype was specified in section 2.5. This section lists what was deliberately excluded from it, the known algorithmic limits, and the experimental limits of the reference dataset.

11.1 Functional out-of-scope

- **Real Jira data** not yet ingested: the move to production requires adaptation to the Jira Agile@Scale format as well as authorization management.

- **Multi-ART** not supported: planning is done on one ART at a time. Multi-project exists (epic → feature) but not simultaneous multi-ART, addressed in the literature [16].
- **Real-time replanning** not handled: the system operates in *batch* mode, continuous *replanning* during a PI belongs to another class of problem.
- **Reinforcement learning** not implemented: the NSGA-II + Q-Learning hybridization of [4] remains to be explored.

11.2 Algorithmic limits

- **NSGA-II with crowding distance**: NSGA-III [9, 11] would be more suited for 6 objectives.
- **No global end-of-PI buffer**. As it stands, we insert a small buffer *after each risky story* to absorb its possible delays (this is what is called a *feeding buffer*). Goldratt's method additionally recommends a **single large buffer at the very end of the PI** (the *project buffer*), which serves as an aggregated safety net for the whole ART. We have not yet put it in place: the RTE therefore drives dozens of small local buffers instead of a single global buffer, which makes it more difficult to see whether the PI as a whole is in danger.
- **span_factor = 2 is a lower bound**: the Kingman law gives a lead-time that grows hyperbolically beyond 80 % utilization [25]; on high-variance projects, the coefficient underestimates the real lead-time.
- **Instance size not tested**. The five datasets contain about a hundred *stories*, which corresponds to one ART. Nothing has been measured beyond that, and the literature reports precisely that NSGA-II degrades as the number of decision variables grows [5]. An ART twice that size, or several ARTs handled together, would require re-running the campaign before announcing any figure.

11.3 Experimental limits

The threats to the campaign's validity are analysed in section 9 and are not repeated here. Two limits specific to the reference dataset are worth adding.

On `seed=42`, the capacity *gap* remains negative on all skills, so the HITL scenario of the AI removal on a *data* bottleneck does not trigger spontaneously. A `--amplify-bottleneck` mode or an enrichment of the templates with more *data features* is necessary to make the demonstration clinically triggerable. This is all the more notable since the campaign shows that a skill bottleneck **does exist** on that instance, and that it alone fixes the achievable makespan: the detector measures aggregate demand per skill, whereas the real constraint bears on the individual resource that happens to be the only eligible one.

Finally, the comparison covers NSGA-II variants and greedy heuristics, not an exact solver nor the other multi-objective metaheuristics of the literature (NSGA-III, SPEA2, MOEA/D). Positioning SAFESCALE against that state of the art would require reimplementing those algorithms on the same model, which is beyond the scope of this thesis.

12 Points to explore

Beyond the delivered version, six extension leads are identified. They are deliberately technical, each addresses a precise point of the algorithm or the system, but we announce them here without going into the implementation details. The reader interested in the underlying mechanisms can refer to section 3 (state of the art).

12.1 P2: Move to NSGA-III to better handle 6 objectives

In one sentence: replace NSGA-II with a more recent variant, NSGA-III, better equipped to compare schedules on many criteria at the same time.

NSGA-II uses the *crowding distance* to preserve the diversity of the Pareto front. This technique is effective up to three objectives, but loses precision beyond. NSGA-III replaces this mechanism with a system of **reference points** placed on a normalized hyperplane, which explicitly guides the coverage of the front [9, 11, 10]. At six objectives as we have, the gain is documented in the literature [6]; the cost is an additional parameterization (placement of the points), which requires its own study.

12.2 P3: NSGA-II + Q-Learning

In one sentence: let the system *learn* the good start dates, instead of choosing them by a fixed rule.

[4] demonstrates that learning a *timing* policy (choice of the `start_day` of each activity) as a Markov decision process improves the results relative to pure NSGA-II, PSO and ACO. It is the natural complement to our *urgency* seed which currently rests on a fixed heuristic.

12.3 P4: Real Jira data and multi-ART

In one sentence: plug SAFESCALE onto the real Jira data and let it plan several ARTs at a time.

[16] adapts NSGA-II to multi-project with sustainability criteria, filling a gap in the literature. Combined with a Jira Agile@Scale adapter, this work would allow the move from the prototype to an industrial framework.

12.4 P5: Global project buffer

In one sentence: replace the dozens of small local buffers with a single large global end-of-PI buffer, easier to drive.

Consolidating the local *feeding buffers* of DEC-018 into a global end-of-PI *project buffer* in Goldratt's sense [24] would allow the RTE to drive a single aggregated buffer rather than dozens of local buffers. The algorithmic cost is low but the change of mental model on the user side requires a study.

12.5 P6: Flexible resource profiles

In one sentence: let the system adjust on the fly how many people work on a story, rather than freezing it at planning time.

[15] addresses the case where the amount of resource per activity is determined while the activity runs rather than fixed at planning time. Its useful contribution here is not the algorithm but the schedule generation

schemes that come with it: deciding no longer only the order of activities, but also the combination of people assigned to each. Relevant for resources shared between SAFe teams via the *component teams*.

12.6 P7: Make use of under-activity teams

In one sentence: when a team has free capacity, propose that it anticipate stories from the next PI or come to the aid of an overloaded team.

The field feedback reveals a symmetric case of over-capacity: teams committed below their potential on a given PI. Two levers are to be instrumented in the tool: (i) propose to the RTE to **anticipate stories from the next PI** compatible with the available skills of the under-utilized team; (ii) allow an under-activity team to **take on stories from other overloaded teams**, subject to specialization compatibility. The skill modeling (cf. section 4) and the removal/addition mechanism at step 3 already lay the necessary building blocks; it remains to wire the decision rules and the RTE interface for these two movements.

13 Conclusion

This thesis presents SAFESCALE, a functional prototype for SAFe PI Planning combining six-objective NSGA-II, an orchestrated multi-agent architecture and safety margins inspired by the Critical Chain. The main contribution is not a new algorithm but the assembly of already-proven elements ([3, 1, 22, 25]) in the precise context of SAFe PI Planning, WSJF, sprints, specialized teams, *readiness* status, inter-feature dependencies, a context that the interview with a Product Owner helped to frame.

The decisions table (table 2) links each design choice to its source, an article or a point noted during the PO interview. It is a means, more than a stylistic effect, to be able to justify the system before a jury.

At this stage, the work remains a prototype, but a measured one: the flow runs end to end on the reference dataset, and the experimental campaign (section 9) establishes three things. The schedules produced are feasible; multi-objective optimisation divides capacity violation by six compared with a greedy scheduler, without being able to improve a makespan bounded by a skill bottleneck; and the blocking threshold at step 3 is what stops the system from producing a structurally undeliverable commitment. The campaign also exposes its own limits: of the two improvements made to NSGA-II, only one is demonstrated, and Pareto dominance saturates at six objectives. The qualitative feedback further suggests that the breakdown into five steps with conditional HITL makes the arbitrations legible; that, by contrast, still has to be confirmed by an evaluation with users. The limits (section 11) and the extension leads (section 12) indicate the next steps toward a real use.

A Appendix: Glossary of acronyms

ACO

Ant Colony Optimization, optimization by ant colonies.

ART

Agile Release Train, grouping of 5 to 15 teams that deliver together.

BDI

Beliefs, Desires, Intentions, autonomous agent model (not retained here).

CNP

Capacity Negotiation Protocol, step 3 of the flow, capacity negotiation.

CPO

Chief Product Owner, prioritizes the features of his scope.

DAG

Directed Acyclic Graph, directed graph without a cycle, models the dependencies.

EDM

Engineering Delivery Manager, responsible for delivery on the engineering side.

HITL

Human-In-The-Loop, intervention of a human in the decision loop.

LLM

Large Language Model, used here as a translator.

MAS

Multi-Agent System.

ML

Machine Learning.

MOEA

Multi-Objective Evolutionary Algorithm.

MS-RCMPSP

Multi-Skill Resource-Constrained Multi-Project Scheduling Problem, multi-skill and multi-project extension of the RCMPSP.

NSGA-II/III

Non-dominated Sorting Genetic Algorithm, non-dominated sorting genetic algorithm, versions 2 and 3.

PI

Program Increment, delivery cycle of 8 to 12 weeks.

PO

Product Owner, carries the content of a team.

PSO

Particle Swarm Optimization.

RCPSP

Resource-Constrained Project Scheduling Problem, project scheduling problem under resource constraints.

ROAM

Resolved, Owned, Accepted, Mitigated, SAFe risk management board.

RTE

Release Train Engineer, facilitator of the ART.

SAFe

Scaled Agile Framework.

SM

Scrum Master, the team's facilitator.

SP

Story Points, relative unit of effort estimation.

WSJF

Weighted Shortest Job First, prioritization score: value over effort.

B Appendix: Glossary of technical terms

Optimization and evolutionary algorithms

Evolutionary algorithm

Algorithm that imitates natural evolution (crossover, mutation, selection) on a population of solutions.

Genetic algorithm

An evolutionary algorithm that evolves a population through crossover and mutation, by analogy with natural selection.

Metaheuristic

General method that finds very good solutions without guaranteeing the exact optimum.

Individual / chromosome

A candidate solution, here, a complete schedule.

Gene

Smallest element of a chromosome; here, the pair (*story*, resource).

Crossover

Combination of two parent solutions to produce a child.

Mutation

Random modification of a part of a solution.

Elitism

Systematic preservation of the best solutions from one generation to the next.

Pareto front

Set of the best compromises: no solution of the front can be improved on one criterion without degrading another.

Dominance

A dominates B if A is at least as good everywhere and strictly better somewhere.

Crowding distance

Measure favoring well-distributed solutions on the front.

Local search

Step-by-step improvement of a solution by small modifications.

Memetic algorithm

Evolutionary algorithm augmented with a local search.

Seed (initialization)

Smart starting solution injected into the initial population.

Seed (random seed)

Number fixing the randomness; at a fixed seed, the program is reproducible.

Hamming distance

Number of positions where two solutions differ.

NP-hard

Problem for which no known algorithm gives the optimum in reasonable time at large size.

Scheduling**Lead-time**

Elapsed time between the start and the end of a task, waiting included, as opposed to effective working time.

Makespan

Total duration of the project, from the start of the first story to the end of the last.

Critical path

Longest chain of dependencies; fixes the minimum duration of the project.

Topological sort

Ordering of a DAG respecting all the dependencies.

Decoding

Transformation of a chromosome into a concrete schedule (start/end dates).

Weighted tardiness

Delay of a delivery multiplied by the importance of the delayed element.

Bottleneck

Skill too much in demand relative to the number of people who master it.

SAFe and agility**Epic / Feature / Story**

Levels of work breakdown; the *story* is the planning unit.

Story point

Relative unit of effort estimation of a story.

Velocity

Story points completed per team and per sprint.

Poker planning

Collective estimation of the effort of the stories.

Readiness

Level of preparation of a feature before planning.

Sprint / cadence

Short iteration (3 weeks); the cadence is the regular rhythm of the ceremonies.

Component team

Cross-cutting team; exception to the principle “one team = one feature”.

Architecture and modeling**Orchestration / emergence**

Orchestration: a central conductor drives the agents. Emergence: autonomous agents without a leader.

Conditional HITL

The human is consulted only if a problem is detected.

Mock

Fake version of a component for reproducible tests.

Effort-duration / calendar-duration

The real effort (person-days) against the time actually elapsed

(lead-time).

Span factor

Coefficient transforming the effort-duration into calendar-duration (pivot 2.0).

Kingman's law

Beyond $\sim 80\%$ utilization, the waiting delays explode, justifies the 20% buffer.

Critical Chain

Goldratt's method: group the margins into targeted buffers rather than inflate each estimate.

Feeding buffer

Time buffer inserted after a risky story.

Deterministic risk score

Score 0-1 computed by a fixed formula (same inputs $\Rightarrow$ same score).

Evaluation**Synthetic dataset**

Artificially generated dataset, with dimensions calibrated on a real case.

Reproducibility

Property of an experiment that gives back the same result at each execution.

Hypervolume

Volume of the objective space covered by a Pareto front; larger = better. Only meaningful relative to a fixed reference box, shared by the fronts being compared.

Spread

Measure of the good distribution of the solutions on the front.

IGD+

Mean distance from a reference front to the evaluated front, counting only unfavourable gaps; lower = better.

C-metric

Share of one front's solutions covered by at least one solution of another front. Not a symmetric indicator.

Ablation

Comparison protocol that disables one contribution at a time, to measure what each of them actually brings.

Effect size

Measure of the magnitude of a gap, independent of the number of observations; here Vargha-Delaney's A_{12} .

Lower bound

Value no solution can go below, computed analytically; used to situate a result when the optimum is unknown.

Composite score

Single mark combining the six objectives normalized and weighted.

Technical debt

Known corrective work, deliberately deferred and documented.

References

- [1] X. Wen, Q. Song, Y. Qian *et al.*, “Effective Improved NSGA-II Algorithm for Multi-Objective IPPS”, *Mathematics*, MDPI, 11(16):3523, 2023.
<https://ideas.repec.org/a/gam/jmathe/v11y2023i16p3523-d1217570.html>
- [2] J. Wang, L. Xu, S. Sun *et al.*, “Multi-objective optimization using improved NSGA-II for IPPS in a machining job shop for large-size valve”, *PLoS ONE*, 19(6):e0306024, 2024.
<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0306024>
- [3] S. C. Burmeister, D. Guericke, G. Schryen, “A memetic NSGA-II for the multi-objective flexible job shop scheduling problem with real-time energy tariffs”, *Flexible Services and Manufacturing Journal*, Springer, 36(4):1530-1570, 2024.
<https://link.springer.com/article/10.1007/s10696-023-09517-7>
- [4] H. Yang, Z. Wang, Y. Gao, W. Zhou, “Bi-objective multi-mode resource-constrained multi-project scheduling using combined NSGA-II and Q-learning”, *Applied Soft Computing*, 152:111201, 2024.
<https://www.sciencedirect.com/science/article/abs/pii/S156849462301219X>

- [5] A. J. Nebro, J. Galeano-Brajones, F. Luna, C. A. Coello Coello, "Is NSGA-II Ready for Large-Scale Multi-Objective Optimization?", *Mathematical and Computational Applications*, MDPI, 27(6):103, 2022.
<https://www.mdpi.com/2297-8747/27/6/103>
- [6] Z. Zhang, "Multi-objective optimization method for building energy-efficient design based on multi-agent-assisted NSGA-II", *Energy Informatics*, Springer, 7(1), 2024.
<https://link.springer.com/article/10.1186/s42162-024-00394-4>
- [7] S. Verma, M. Pant, "A Comprehensive Review on NSGA-II for Multi-Objective Combinatorial Optimization Problems", IEEE Xplore, 2021.
<https://ieeexplore.ieee.org/document/9393947/>
- [8] Wang Yahui, Shi Ling, Zhang Cai *et al.*, "NSGA-II algorithm and application for multi-objective flexible workshop scheduling", *Sage Journals*, 2020.
<https://journals.sagepub.com/doi/full/10.1177/1748302620942467>
- [9] K. Deb, H. Jain, "An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point-Based Nondominated Sorting Approach, Part I", *IEEE TEVC*, 18(4):577-601, 2014.
<https://ieeexplore.ieee.org/document/6600851>
- [10] Z. Ling, Y. Chen, "Research on Multi-Objective Optimization of Project Scheduling based on NSGA-III: Trade-off among Project Duration, Resource Allocation, and Robustness", *International Core Journal of Engineering*, 11(4):413-422, 2025.
<https://bcpublication.org/index.php/ICJE/article/view/8360>
- [11] V. H. S. Pham, N. T. P. Hoang, "Advancing multi-objective optimization in resource management using the NSGA-III algorithm", *Canadian Journal of Civil Engineering*, 52(7): 1319-1335, 2025.
<https://cdnsiencepub.com/doi/10.1139/cjce-2024-0298>
- [12] V. Valls, F. Ballestín, S. Quintanilla, "A hybrid genetic algorithm for the resource-constrained project scheduling problem", *European Journal of Operational Research*, 185(2):495-508, 2008.
<https://www.sciencedirect.com/science/article/abs/pii/S0377221707000616>
- [13] E. Goncharov, "A hybrid heuristic algorithm for the resource-constrained project scheduling problem", arXiv:2502.18330, 2025.
<https://arxiv.org/abs/2502.18330>
- [14] R. Torba, S. Dauzère-Pérès, C. Yugma, C. Gallais, J. Pouzet, "Solving a real-life multi-skill resource-constrained multi-project scheduling problem", *Annals of Operations Research*, Springer, 338(1):69-114, 2024.
<https://link.springer.com/article/10.1007/s10479-023-05784-7>
- [15] X. Luo, S. Guo, B. Du, X. Luo, J. Guo, "Multi-Objective Multi-Skill Resource-Constrained Project Scheduling Considering Flexible Resource Profiles", *MDPI Applied Sciences*, 14(5):1921, 2024.
<https://www.mdpi.com/2076-3417/14/5/1921>
- [16] M. A. Lostumbo, M. Bara Iniesta, L. Calvet, "Sustainable Optimization of Construction Project Portfolio Management Through a Data-Driven Approach", in *Decision Sciences*, Lecture Notes in Computer Science, Springer, 2025, pp. 60-71.
https://doi.org/10.1007/978-3-031-78238-1_6
- [17] H. Yang, Y. Yuan, S. Ye, L. Lin, "Fuzzy Resource Constrained Project Scheduling Optimization with Hybrid Multiobjective Genetic Algorithm", *ACM ICBC*, pp. 280-285, 2021.
<https://dl.acm.org/doi/10.1145/3448748.3448793>
- [18] S. Rodríguez-Ballesteros, J. Alcaraz, L. Anton-Sanchez, "Multi-mode RCPSP with time-dependent resource costs and capacities: A bi-objective approach", *Expert Systems with Applications*, 306:130955, 2026.
<https://www.sciencedirect.com/science/article/pii/S0957417425045701>
- [19] R. Pellerin *et al.*, "A survey of hybrid metaheuristics for the resource-constrained project scheduling problem", *EJOR*, 280(2), 2020.
<https://www.sciencedirect.com/science/article/abs/pii/S0377221719300475>
- [20] W. Herroelen, B. De Reyck, E. Demeulemeester, "Resource-constrained project scheduling: A survey of recent developments", *Computers & Operations Research*, 25(4), 1998.
<https://www.sciencedirect.com/science/article/abs/pii/S0305054897000551>
- [21] Scaled Agile Inc., *SAFe 6.0 Framework, PI Planning*.
<https://framework.scaledagile.com/pi-planning>
- [22] W. Zhang, G. Xiao, M. Gen, H. Geng, X. Wang, M. Deng, G. Zhang, "Enhancing multi-objective evolutionary algorithms with machine learning for scheduling problems: recent advances and survey", *Frontiers in Industrial Engineering*, 2, 2024.
<https://www.frontiersin.org/journals/industrial-engineering/articles/10.3389/fieng.2024.1337174/full>
- [23] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, "A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II", *IEEE Transactions on Evolutionary Computation*, 6(2):182-197, 2002.
<https://ieeexplore.ieee.org/document/996017>
- [24] E. M. Goldratt, *Critical Chain*, North River Press, Great Barrington (MA), 1997. ISBN 978-0-88427-153-6.
- [25] D. G. Reinertsen, *The Principles of Product Development Flow: Second Generation Lean Product Development*, Celeritas Publishing, Redondo Beach (CA), 2009. ISBN 978-1-935401-00-1.
- [26] J. Blazewicz, J. K. Lenstra, A. H. G. Rinnooy Kan, "Scheduling subject to resource constraints: classification and complexity", *Discrete Applied Mathematics*, 5:11-24, 1983.
<https://www.sciencedirect.com/science/article/pii/0166218X83900124>
- [27] E. Zitzler, L. Thiele, "Multiobjective Evolutionary Algorithms: A Comparative Case Study and the Strength Pareto Approach", *IEEE Transactions on Evolutionary Computation*, 3(4):257-271, 1999.
<https://ieeexplore.ieee.org/document/797969>
- [28] H. Ishibuchi, H. Masuda, Y. Tanigaki, Y. Nojima, "Modified Distance Calculation in Generational Distance and Inverted Generational Distance", *Evolutionary Multi-Criterion Optimization (EMO 2015)*, LNCS 9019:110-125, 2015.
https://link.springer.com/chapter/10.1007/978-3-319-15892-1_8
- [29] A. Vargha, H. D. Delaney, "A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong", *Journal of Educational and Behavioral Statistics*, 25(2):101-132, 2000.
<https://journals.sagepub.com/doi/10.3102/10769986025002101>
- [30] A. Arcuri, L. Briand, "A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering", *33rd International Conference on Software Engineering (ICSE)*, 1-10, 2011.
<https://ieeexplore.ieee.org/document/6032496>